

Operating Systems – Midterm

April 15, 2014

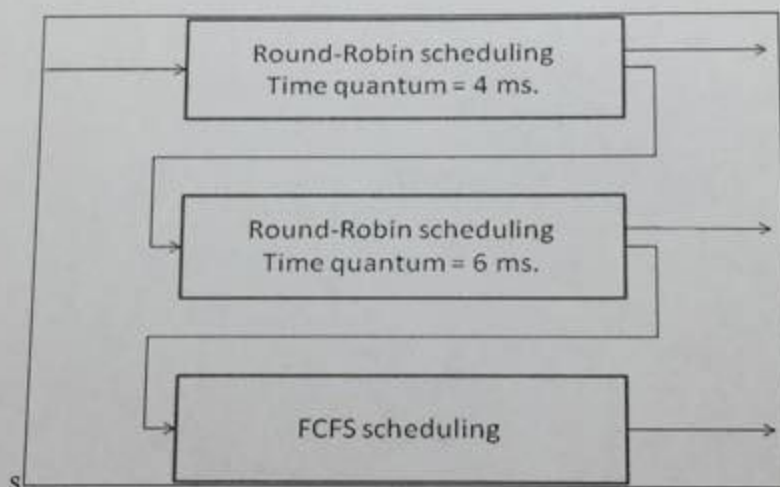
Total: 100 points

1. Hardware devices signal interrupts when important events occur (e.g., hard-disk data are available):
 - (a) (5 points) Show the advantage of interrupt-based operations over busy-waiting.
 - (b) (10 points) Describe the process of hardware interrupt handling. You **need to** mention *device controller*, *interrupt-request line*, *state saving*, *interrupt vector*, *interrupt-specific handler*.
 - (c) (5 points) Explain the benefit of using interrupt vector.
2. (5 points) What is the difference between asymmetric multiprocessing and symmetric multiprocessing?
3. (5 points) How do operating systems support new devices?
4. System calls:
 - (a) (10 points) Describe how a system call is invoked using an API function? You **need to** mention *software interrupt*, *dual-mode*, *system call number (index)*, and *table of code pointer*.
 - (b) (5 points) Explain why programmers prefer programming according to API functions rather than invoking system calls directly.
 - (c) (5 points) Describe a method to pass system call parameters to the operating system.
5. (5 points) A process in memory generally consists of four major components. What are the components and what is the use of each component?
6. Multithread Models:
 - (a) (5 points) Explain the problem of many-to-many thread model without using scheduler activations.
 - (b) (10 points) Show how scheduler activations work when a user thread is making a blocking system call. You **HAVE TO** mention *upcall handler*, *scheduler*, and *thread library*.

7. (10 points) Show the four situations under which a preemptive scheduling algorithm takes place. And what is the difference between preemptive scheduling and non-preemptive scheduling?
8. Scheduling – Consider the following two processes P_1 and P_2 , each of which consists of two CPU bursts and one I/O burst. Assume that P_1 and P_2 arrive at time 0 and 2 (millisecond) respectively:

P_1 :	CPU burst: 8 milliseconds.	I/O burst: 4 milliseconds.	CPU burst: 8 milliseconds.
P_2 :	CPU burst: 2 milliseconds.	I/O burst: 2 milliseconds.	CPU burst: 16 milliseconds.

- (a) (5 points) Draw a Gantt chart illustrating the execution of the processes using the following **preemptive multilevel feedback-queue scheduling algorithm**. **Note that a high-level process will preempt a low-level process and a process will be scheduled to a lower level if it is preempted.**



- (b) (5 points) What is the average waiting time for the scheduling algorithm?

9. (10 points) Complete the following program which calculates $\sum_{n=1}^N n!$. The value N is provided by users in the command line, i.e., $N = \text{atoi}(\text{argv}[1])$:

Hint: The size of an integer is 4 bytes. The program needs to create N child processes - one for each factorial calculation (i.e., $n!$). The child processes save the results in a shared integer memory and the parent process prints the final outcome. To prevent inconsistent results, the parent needs to wait for each child process.

```
int main(int argc, char *argv[])
{
    int N = atoi(argv[1]);
    pid_t pid;
    int segment_id;
    int *shared_memory;

    // allocate and attach a shared memory segment

    for( ) // create child processes
    {

        if( ) {
            fprintf(stderr, "Fork Failed");
            exit(-1);

        } else if( ) { // child part
            // do factorial calculation and save it
            // then detach shared memory and exit

        } else { // parent part

        }
    }

    printf("the result is %d\n", *shared_memory);

    // detach and remove the shared memory segment

    return 0;
}
```